



Задание к экзамену

Задание: разработка системы комплектации заказов на складе (на Java).

Цель: продемонстрировать навыки применения принципов ООП (абстракция, инкапсуляция, наследование, полиморфизм), работы с интерфейсами, обобщениями (Generics), коллекциями, функциональным программированием (лямбда-выражения, Stream API), условными конструкциями, циклами и проектирования консольного интерфейса.

Описание:

Разработать консольное приложение для склада одного вида товара. Заказы поступают поштучно и упаковками. Запас ограничен; заказ можно выполнить только целиком. Порядок обработки и аналитический запрос определяются индивидуальным вариантом.



| Программирование

Требования к реализации (Обязательная часть на оценку до «Хорошо» включительно):

Только стандартная библиотека Java; сторонние библиотеки не использовать. Конструктор при недопустимых параметрах выбрасывает `IllegalArgumentException`. Ошибочный ввод и отклонённые операции не меняют существующие данные; `Main` сообщает об ошибке и продолжает работу.

Абстракция и Интерфейсы:

Определить интерфейс `StockDemand` с методом `int getRequiredUnits()`, возвращающим потребность в единицах товара.

Создать абстрактный класс `Order`, реализующий `StockDemand`.

- Свойства (private): `id` (int — ID заказа), `customerId` (int — ID клиента), `priority` (int, от 1 до 5; 5 — высший приоритет), `fulfilled` (boolean — выполнен ли заказ; первоначально false). ID — положительные числа.
- Методы: конструктор, геттеры, `void markFulfilled()`, `String getDetails()` — сведения о заказе, потребности и состоянии. ID, приоритет и параметры заказа после создания не изменяются.

Наследование и Инкапсуляция:

Создать два класса-наследника. Уникальные поля — private final; доступ через геттеры. Каждый класс реализует `getRequiredUnits()`.

- `UnitOrder` (Поштучный заказ): `quantity` (int, от 1 до 1000). Потребность равна `quantity`.
- `PackOrder` (Заказ упаковками): `packCount` и `packSize` (int, каждый от 1 до 1000). Потребность равна `packCount × packSize` единиц товара.

Некорректные значения должны отклоняться. Упаковки одного заказа имеют одинаковый размер.

Обобщения (Generics) и Функциональные интерфейсы (Core-механика):



| Программирование

Создать `WarehouseManager<T extends Order>`. В индивидуальной логике использовать `Predicate` для проверки условий выполнения и `Comparator` для задания порядка. Потребность заказа вычисляется полиморфно.

Коллекции и Главный менеджер:

Менеджер хранит `private List<T> orders` и `private int stock` — текущий остаток в единицах товара, от 0 до 1 000 000. Остаток задаётся конструктором. В `Main` используется `WarehouseManager<Order>`.

- Методы:
- `boolean add(T order)` — добавить новый невыполненный заказ. При `null`, повторном `id` или `fulfilled = true` вернуть `false` без изменения данных. При успехе вернуть `true`; остаток и ранее выполненные заказы не изменяются.
- `boolean restock(int amount)` — пополнить запас на `amount` единиц, где `amount` от 1 до 1 000 000. При недопустимом значении или превышении максимального остатка вернуть `false` без изменений; иначе — `true`.
- `List<T> process()` — обработать ожидающие заказы по индивидуальному варианту, обновить `stock` и `fulfilled`; вернуть только заказы, выполненные этим вызовом, в порядке обработки.
- `int getStock()` — вернуть текущий остаток; `Stream<T> getAnalyticsStream()` — вернуть поток всех заказов для аналитики.

Консольный интерфейс и циклы:

В `Main` реализовать меню через `Scanner`, цикл `while` или `do-while` и `switch`:

1. Добавить заказ выбранного типа.
2. Показать все заказы и текущий остаток.
3. Пополнить запас товара.
4. Обработать заказы и вывести выполненные этим запуском.
5. Выполнить аналитический запрос своего варианта.
6. Выход из программы.

Начальный остаток и заказы задать в коде. Некорректный ввод не должен завершать программу аварийно.



| Программирование

Начальные данные:

Начальный stock = 15; у всех заказов fulfilled = false. Записи ниже: id, customerId, priority, затем quantity для поштучного заказа либо packCount и packSize для упаковок. Данные создать в указанном порядке.

UnitOrder: (1, 1, 5, 9); (4, 2, 2, 3); (5, 3, 4, 5); (8, 5, 1, 1); (9, 2, 1, 2); (11, 1, 1, 2).

PackOrder: (2, 1, 4, 2, 3); (3, 2, 5, 2, 2); (6, 4, 3, 2, 5); (7, 3, 4, 1, 2); (10, 2, 1, 1, 3).

Бонусное задание (Обязательно для получения оценки «Отлично»):

Работа с файлами и Сериализация:

Реализовать сохранение остатка и списка заказов со всеми полями в warehouse_state.dat при выходе и загрузку при старте. Использовать ObjectOutputStream, ObjectInputStream и Serializable. Если файла нет или он не читается, использовать начальные данные; об ошибке чтения сообщить пользователю. Выполненные заказы после загрузки не должны выполняться повторно. При ошибке сохранения сообщить пользователю, не выводить сообщение об успешном сохранении и вернуться в меню с текущими данными.



| Программирование

Структура проекта:

```
└─ src
    │
    │ └─ interfaces
    │     └─ StockDemand.java
    │
    │ └─ models
    │     │
    │     │ └─ Order.java
    │     │
    │     │ └─ UnitOrder.java
    │     │
    │     └─ PackOrder.java
    │
    │ └─ logic
    │     └─ WarehouseManager.java
    │
    └─ main
        └─ Main.java
```



| Программирование

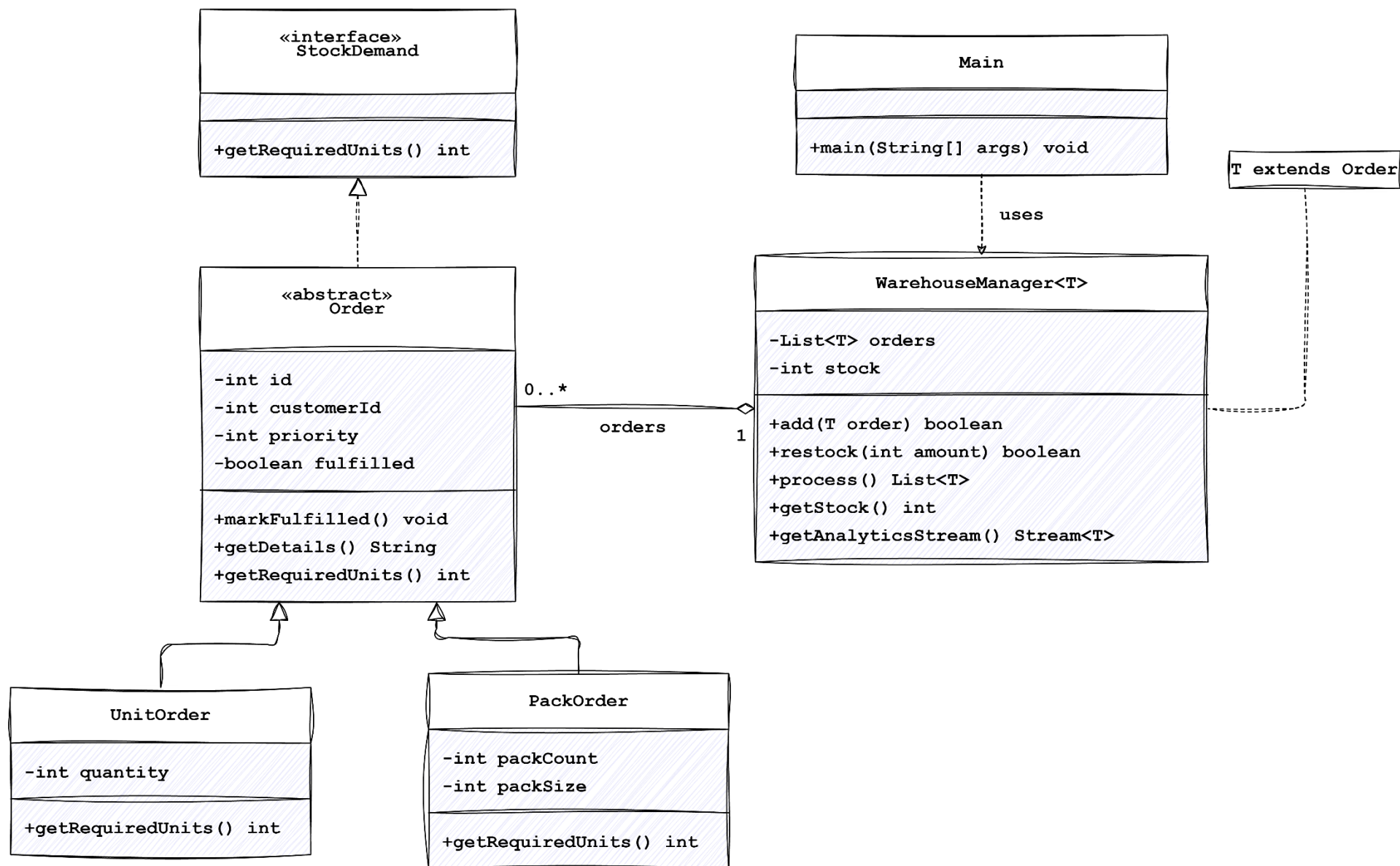
Иерархия классов и UML Диаграмма ([ссылка](#))

Легенда UML-диаграммы (шпаргалка):

- <<interface>> — обозначает, что это интерфейс.
- <<abstract>> — обозначает, что класс абстрактный (его нельзя создать напрямую через оператор new).
- + (плюс) — модификатор доступа public.
- - (минус) — модификатор доступа private (используется для инкапсуляции данных).
- # (решетка) — модификатор доступа protected (поле или метод доступны наследникам данного класса).
- ~T~ — обозначает использование обобщений (Generics), в Java-коде пишется как <T>.
- Пунктирная линия с пустым треугольником (<|..) — реализация интерфейса (implements).
- Сплошная линия с пустым треугольником (<|--) — наследование класса (extends).
- Сплошная линия с ромбом на конце (o--) — отношение агрегации (означает, что класс содержит внутри себя коллекции объектов).
- Пунктирная стрелка uses — использование менеджера классом Main.



Программирование





Варианты индивидуальных заданий

Инструкция: реализовать только свой вариант из таблицы — обработку заказов в пункте меню 4 и аналитику в пункте 5.

Общие правила обработки:

- Суммарную потребность нескольких заказов вычислять в long. Ограничение stock относится к фактическому остатку, а не к сумме потребностей.
- Ожидающим считается заказ с fulfilled = false. Выполненные заказы остаются в системе, но товар по ним повторно не списывается. Один клиент может иметь несколько заказов.
- Заказ выполняется только целиком; stock уменьшается ровно на его потребность. При недостатке товара или невыполнении условия варианта заказ остаётся ожидающим, остаток не изменяется, рассмотрение остальных продолжается. Для группового варианта действуют дополнительные условия таблицы.
- В пределах одного запуска каждый ожидающий заказ рассматривается не более одного раза. При равенстве всех указанных критериев преимущество имеет меньший id заказа. Пустая коллекция или нулевой запас дают пустой результат.

Аналитический запрос выполняется исключительно через Stream API, без циклов for/while, по текущим данным и не изменяет их. Ограничения правила обработки при аналитике не применяются, если прямо не указаны. Пустая выборка даёт пустой результат либо 0 для числового итога.



Программирование

№ варианта	Правило обработки	Аналитика
Вариант 1	Приоритет и малый объём: Порядок: priority по убыванию, потребность по возрастанию, id по возрастанию. За запуск выполнить не более двух заказов каждого клиента; невыполненный заказ не расходует лимит.	Вычислить общий дефицит товара для всех ожидающих заказов. Если запаса хватает, вернуть 0. Вместимость склада, предел одного пополнения и ограничения правила обработки в этом расчёте не учитывать. Результат — long.
Вариант 2	Неприкосновенный резерв: Сохранить не менее четверти запаса на начало запуска, округлённой вверх до целого. Порядок: priority по убыванию, потребность по убыванию, id по возрастанию.	Получить Map<Integer, Long>: customerId → общая потребность его ожидающих заказов. Клиентов без ожидающих заказов не включать.
Вариант 3	Один заказ клиенту: За один запуск выполнить не более одного заказа каждого клиента. Порядок: priority по убыванию, id по возрастанию. Невыполненный заказ не расходует право клиента.	Найти ожидающий заказ, который помещается в текущий запас. Выбрать наибольший priority, затем наибольшую потребность, затем меньший id. Результат — Optional<Order>.
Вариант 4	Клиент целиком: Выполнить все ожидающие заказы клиента вместе либо ни одного. Клиенты упорядочены по суммарной потребности ожидающих заказов, затем по customerId — по возрастанию. Внутри группы — по id.	Вывести по возрастанию ID клиентов, у которых одновременно есть выполненный и ожидающий заказы. Повторных ID быть не должно.



| Программирование

№ варианта	Правило обработки	Аналитика
Вариант 5	Чередование типов: Рассматривать UnitOrder и PackOrder поочерёдно, начиная с UnitOrder. Внутри типа: priority по убыванию, id по возрастанию. Пропуск заказа расходует ход типа. Если один тип исчерпан, рассматривать другой.	Получить Map<String, Long>: имя класса заказа → количество единиц товара по выполненным заказам этого типа. Включить оба типа, при отсутствии — 0.
Вариант 6	Выравнивание обслуживания: Вначале заказы клиентов с меньшим числом выполненных заказов на начало запуска; далее priority по убыванию и id по возрастанию. Число прежних выполнений фиксируется на весь запуск.	Среди клиентов с не менее чем двумя заказами найти клиента с наибольшей долей выполненных заказов в общем числе его заказов. При равенстве — меньший customerId. Результат — Optional<Integer>.
Вариант 7	Приоритет и малый объём: Порядок: priority по убыванию, потребность по возрастанию, id по возрастанию. За запуск выполнить не более двух заказов каждого клиента; невыполненный заказ не расходует лимит.	Получить Map<Integer, Long>: customerId → общая потребность его ожидающих заказов. Клиентов без ожидающих заказов не включать.
Вариант 8	Неприкосновенный резерв: Сохранить не менее четверти запаса на начало запуска, округлённой вверх до целого. Порядок: priority по убыванию, потребность по убыванию, id по возрастанию.	Найти ожидающий заказ, который помещается в текущий запас. Выбрать наибольший priority, затем наибольшую потребность, затем меньший id. Результат — Optional<Order>.



Программирование

№ варианта	Правило обработки	Аналитика
Вариант 9	Один заказ клиенту: За один запуск выполнить не более одного заказа каждого клиента. Порядок: priority по убыванию, id по возрастанию. Невыполненный заказ не расходует право клиента.	Вывести по возрастанию ID клиентов, у которых одновременно есть выполненный и ожидающий заказы. Повторных ID быть не должно.
Вариант 10	Клиент целиком: Выполнить все ожидающие заказы клиента вместе либо ни одного. Клиенты упорядочены по суммарной потребности ожидающих заказов, затем по customerId — по возрастанию. Внутри группы — по id.	Получить Map<String, Long>: имя класса заказа → количество единиц товара по выполненным заказам этого типа. Включить оба типа, при отсутствии — 0.
Вариант 11	Чередование типов: Рассматривать UnitOrder и PackOrder поочередно, начиная с UnitOrder. Внутри типа: priority по убыванию, id по возрастанию. Пропуск заказа расходует ход типа. Если один тип исчерпан, рассматривать другой.	Среди клиентов с не менее чем двумя заказами найти клиента с наибольшей долей выполненных заказов в общем числе его заказов. При равенстве — меньший customerId. Результат — Optional<Integer>.
Вариант 12	Выравнивание обслуживания: Вначале заказы клиентов с меньшим числом выполненных заказов на начало запуска; далее priority по убыванию и id по возрастанию. Число прежних выполнений фиксируется на весь запуск.	Вычислить общий дефицит товара для всех ожидающих заказов. Если запаса хватает, вернуть 0. Вместимость склада, предел одного пополнения и ограничения правила обработки в этом расчёте не учитывать. Результат — long.



Программирование

№ варианта	Правило обработки	Аналитика
Вариант 13	Приоритет и малый объём: Порядок: priority по убыванию, потребность по возрастанию, id по возрастанию. За запуск выполнить не более двух заказов каждого клиента; невыполненный заказ не расходует лимит.	Найти ожидающий заказ, который помещается в текущий запас. Выбрать наибольший priority, затем наибольшую потребность, затем меньший id. Результат — Optional<Order>.
Вариант 14	Неприкосновенный резерв: Сохранить не менее четверти запаса на начало запуска, округлённой вверх до целого. Порядок: priority по убыванию, потребность по убыванию, id по возрастанию.	Вывести по возрастанию ID клиентов, у которых одновременно есть выполненный и ожидающий заказы. Повторных ID быть не должно.
Вариант 15	Один заказ клиенту: За один запуск выполнить не более одного заказа каждого клиента. Порядок: priority по убыванию, id по возрастанию. Невыполненный заказ не расходует право клиента.	Получить Map<String, Long>: имя класса заказа → количество единиц товара по выполненным заказам этого типа. Включить оба типа, при отсутствии — 0.
Вариант 16	Клиент целиком: Выполнить все ожидающие заказы клиента вместе либо ни одного. Клиенты упорядочены по суммарной потребности ожидающих заказов, затем по customerId — по возрастанию. Внутри группы — по id.	Среди клиентов с не менее чем двумя заказами найти клиента с наибольшей долей выполненных заказов в общем числе его заказов. При равенстве — меньший customerId. Результат — Optional<Integer>.



Программирование

№ варианта	Правило обработки	Аналитика
Вариант 17	Чередование типов: Рассматривать UnitOrder и PackOrder поочерёдно, начиная с UnitOrder. Внутри типа: priority по убыванию, id по возрастанию. Пропуск заказа расходует ход типа. Если один тип исчерпан, рассматривать другой.	Вычислить общий дефицит товара для всех ожидающих заказов. Если запаса хватает, вернуть 0. Вместимость склада, предел одного пополнения и ограничения правила обработки в этом расчёте не учитывать. Результат — long.
Вариант 18	Выравнивание обслуживания: Вначале заказы клиентов с меньшим числом выполненных заказов на начало запуска; далее priority по убыванию и id по возрастанию. Число прежних выполнений фиксируется на весь запуск.	Получить Map<Integer, Long>: customerId → общая потребность его ожидающих заказов. Клиентов без ожидающих заказов не включать.
Вариант 19	Приоритет и малый объём: Порядок: priority по убыванию, потребность по возрастанию, id по возрастанию. За запуск выполнить не более двух заказов каждого клиента; невыполненный заказ не расходует лимит.	Вывести по возрастанию ID клиентов, у которых одновременно есть выполненный и ожидающий заказы. Повторных ID быть не должно.
Вариант 20	Неприкосновенный резерв: Сохранить не менее четверти запаса на начало запуска, округлённой вверх до целого. Порядок: priority по убыванию, потребность по убыванию, id по возрастанию.	Получить Map<String, Long>: имя класса заказа → количество единиц товара по выполненным заказам этого типа. Включить оба типа, при отсутствии — 0.



№ варианта	Правило обработки	Аналитика
Вариант 21	Один заказ клиенту: За один запуск выполнить не более одного заказа каждого клиента. Порядок: priority по убыванию, id по возрастанию. Невыполненный заказ не расходует право клиента.	Среди клиентов с не менее чем двумя заказами найти клиента с наибольшей долей выполненных заказов в общем числе его заказов. При равенстве — меньший customerId. Результат — Optional<Integer>.
Вариант 22	Клиент целиком: Выполнить все ожидающие заказы клиента вместе либо ни одного. Клиенты упорядочены по суммарной потребности ожидающих заказов, затем по customerId — по возрастанию. Внутри группы — по id.	Вычислить общий дефицит товара для всех ожидающих заказов. Если запаса хватает, вернуть 0. Вместимость склада, предел одного пополнения и ограничения правила обработки в этом расчёте не учитывать. Результат — long.
Вариант 23	Чередование типов: Рассматривать UnitOrder и PackOrder поочередно, начиная с UnitOrder. Внутри типа: priority по убыванию, id по возрастанию. Пропуск заказа расходует ход типа. Если один тип исчерпан, рассматривать другой.	Получить Map<Integer, Long>: customerId → общая потребность его ожидающих заказов. Клиентов без ожидающих заказов не включать.
Вариант 24	Выравнивание обслуживания: Вначале заказы клиентов с меньшим числом выполненных заказов на начало запуска; далее priority по убыванию и id по возрастанию. Число прежних выполнений фиксируется на весь запуск.	Найти ожидающий заказ, который помещается в текущий запас. Выбрать наибольший priority, затем наибольшую потребность, затем меньший id. Результат — Optional<Order>.



| Программирование

№ варианта	Правило обработки	Аналитика
Вариант 25	Приоритет и малый объём: Порядок: priority по убыванию, потребность по возрастанию, id по возрастанию. За запуск выполнить не более двух заказов каждого клиента; невыполненный заказ не расходует лимит.	Получить Map<String, Long>: имя класса заказа → количество единиц товара по выполненным заказам этого типа. Включить оба типа, при отсутствии — 0.
Вариант 26	Неприкосновенный резерв: Сохранить не менее четверти запаса на начало запуска, округлённой вверх до целого. Порядок: priority по убыванию, потребность по убыванию, id по возрастанию.	Среди клиентов с не менее чем двумя заказами найти клиента с наибольшей долей выполненных заказов в общем числе его заказов. При равенстве — меньший customerId. Результат — Optional<Integer>.
Вариант 27	Один заказ клиенту: За один запуск выполнить не более одного заказа каждого клиента. Порядок: priority по убыванию, id по возрастанию. Невыполненный заказ не расходует право клиента.	Вычислить общий дефицит товара для всех ожидающих заказов. Если запаса хватает, вернуть 0. Вместимость склада, предел одного пополнения и ограничения правила обработки в этом расчёте не учитывать. Результат — long.
Вариант 28	Клиент целиком: Выполнить все ожидающие заказы клиента вместе либо ни одного. Клиенты упорядочены по суммарной потребности ожидающих заказов, затем по customerId — по возрастанию. Внутри группы — по id.	Получить Map<Integer, Long>: customerId → общая потребность его ожидающих заказов. Клиентов без ожидающих заказов не включать.



№ варианта	Правило обработки	Аналитика
Вариант 29	Чередование типов: Рассматривать UnitOrder и PackOrder поочередно, начиная с UnitOrder. Внутри типа: priority по убыванию, id по возрастанию. Пропуск заказа расходует ход типа. Если один тип исчерпан, рассматривать другой.	Найти ожидающий заказ, который помещается в текущий запас. Выбрать наибольший priority, затем наибольшую потребность, затем меньший id. Результат — Optional<Order>.
Вариант 30	Выравнивание обслуживания: Вначале заказы клиентов с меньшим числом выполненных заказов на начало запуска; далее priority по убыванию и id по возрастанию. Число прежних выполнений фиксируется на весь запуск.	Вывести по возрастанию ID клиентов, у которых одновременно есть выполненный и ожидающий заказы. Повторных ID быть не должно.
Вариант 31	Приоритет и малый объём: Порядок: priority по убыванию, потребность по возрастанию, id по возрастанию. За запуск выполнить не более двух заказов каждого клиента; невыполненный заказ не расходует лимит.	Среди клиентов с не менее чем двумя заказами найти клиента с наибольшей долей выполненных заказов в общем числе его заказов. При равенстве — меньший customerId. Результат — Optional<Integer>.
Вариант 32	Неприкосновенный резерв: Сохранить не менее четверти запаса на начало запуска, округлённой вверх до целого. Порядок: priority по убыванию, потребность по убыванию, id по возрастанию.	Вычислить общий дефицит товара для всех ожидающих заказов. Если запаса хватает, вернуть 0. Вместимость склада, предел одного пополнения и ограничения правила обработки в этом расчёте не учитывать. Результат — long.



№ варианта	Правило обработки	Аналитика
Вариант 33	Один заказ клиенту: За один запуск выполнить не более одного заказа каждого клиента. Порядок: priority по убыванию, id по возрастанию. Невыполненный заказ не расходует право клиента.	Получить Map<Integer, Long>: customerId → общая потребность его ожидающих заказов. Клиентов без ожидающих заказов не включать.
Вариант 34	Клиент целиком: Выполнить все ожидающие заказы клиента вместе либо ни одного. Клиенты упорядочены по суммарной потребности ожидающих заказов, затем по customerId — по возрастанию. Внутри группы — по id.	Найти ожидающий заказ, который помещается в текущий запас. Выбрать наибольший priority, затем наибольшую потребность, затем меньший id. Результат — Optional<Order>.
Вариант 35	Чередование типов: Рассматривать UnitOrder и PackOrder поочередно, начиная с UnitOrder. Внутри типа: priority по убыванию, id по возрастанию. Пропуск заказа расходует ход типа. Если один тип исчерпан, рассматривать другой.	Вывести по возрастанию ID клиентов, у которых одновременно есть выполненный и ожидающий заказы. Повторных ID быть не должно.
Вариант 36	Выравнивание обслуживания: Вначале заказы клиентов с меньшим числом выполненных заказов на начало запуска; далее priority по убыванию и id по возрастанию. Число прежних выполнений фиксируется на весь запуск.	Получить Map<String, Long>: имя класса заказа → количество единиц товара по выполненным заказам этого типа. Включить оба типа, при отсутствии — 0.